

Sorting

What is Sorting?

- Taking an array or list of unordered elements and turning it into a sorted array or list.
- Arranging elements done in a certain order (Ascending or Descending).
- `sort()`: default function in java, sorts array in ascending order.
- Ex:

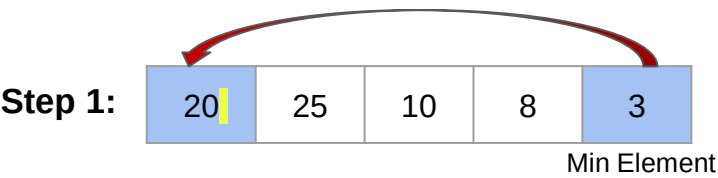
```
1. import java.util.Arrays;
2. public class Sort {
3.     public static void main(String args[]) {
4.         int[] array = {10, 35, 8, 28, -3};
5.         Arrays.sort(array);
6.         System.out.println("Sorted Array in Ascending Order: ");
7.         System.out.println(Arrays.toString(array));
8.     }
9. }
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop
[-3, 8, 10, 28, 35]
```

Selection Sort

- One of the most popular sorting techniques.
- Sorting done on comparison basis.
- Ex: For sorting in ascending order, first element of array compared with rest of the elements.
Minimum value found and swapped with the first element.
- Similarly, second element of array compared with the elements in array after the second element.
Minimum value found and swapped with the second element.
- And so on for the rest of the elements until array is completely sorted.

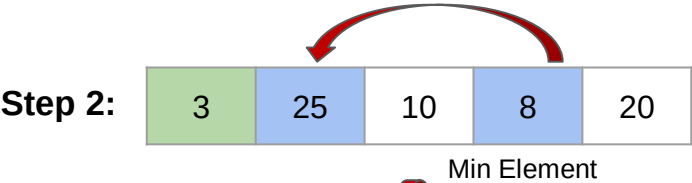
Selection Sort (Contd.)



After Swapping

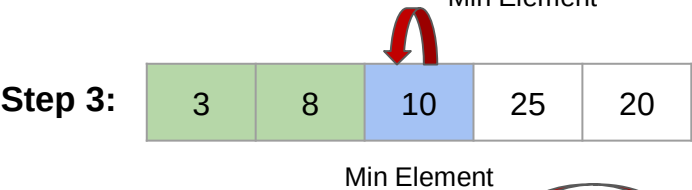
20	25	10	8	3
----	----	----	---	---

3	25	10	8	20
---	----	----	---	----



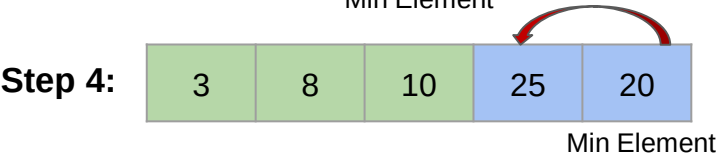
After Swapping

3	8	10	25	20
---	---	----	----	----



Swapped with Own Element

3	8	10	25	20
---	---	----	----	----



After Swapping

3	8	10	20	25
---	---	----	----	----

Final Array

3	8	10	20	25
---	---	----	----	----

Selection Sort

```
1. int[] array = {20, 25, 10, 8, 3};
2. for (int i = 0; i < array.length - 1; i++)
3.     {
4.         int min_index = i;
5.         for (int j = i + 1; j < array.length; j++){
6.             if (array[j] < array[min_index]){
7.                 min_index = j;
8.             }
9.         }
10.        int temp = array[min_index];
11.        array[min_index] = array[i];
12.        array[i] = temp;
13.    }
14. System.out.println("Sorted Array in Ascending Order: ");
15. System.out.println(Arrays.toString(array));
```

Welcome to DrJava. Working directory is
C:\Users\Desktop
[3, 8, 10, 20, 25]

Bubble Sort

- Sorting done by comparing two adjacent elements in an array in each iteration and swapping them.
- Process repeated till array is sorted and swapping not needed.
- At the end of each iteration of outer loop, one element from the last of array sorted.

Bubble Sort (Contd.)

Initial Array:

10	5	20	1
----	---	----	---

Step 1 :

10	5	20	1
----	---	----	---



5	10	20	1
---	----	----	---



No swap

5	10	20	1
---	----	----	---

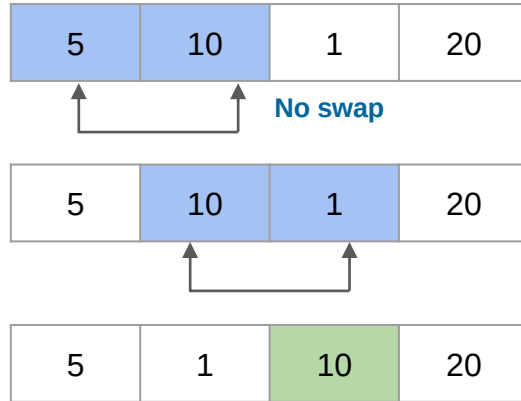


5	10	1	20
---	----	---	----

Maximum element, i.e. 20 sorted at the end of first iteration of outer loop.

Bubble Sort (Contd.)

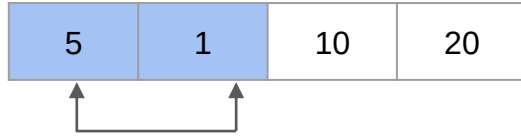
Step 2 :



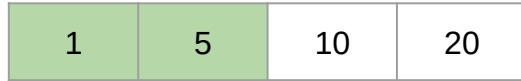
Second maximum element, i.e. 10 sorted at the end of second iteration of outer loop.

Bubble Sort (Contd.)

Step 3 :



Final Array:



Remaining elements sorted at the end of third iteration of outer loop.

Bubble Sort (Contd.)

```
1. import java.util.Arrays;
2. public class BubbleSort {
3.     public static void main(String args[]) {
4.         int[] array = {10, 5, 20, 1};
5.         for (int i = 0; i < array.length - 1; i++){
6.             for (int j = 0; j < array.length - i - 1; j++){
7.                 if (array[j] > array[j + 1]) {
8.                     int temp = array[j];
9.                     array[j] = array[j + 1];
10.                    array[j + 1] = temp;
11.                }
12.            }
13.        }
14.        System.out.println("Sorted Array in Ascending Order:");
15.        System.out.println(Arrays.toString(array));
16.    }
17. }
```

```
Welcome to DrJava. Working directory is
C:\Users\Desktop
[1, 5, 10, 20]
```

THANK YOU!